

TP FINAL NOTÉ - Sécurisation de code vulnérable

CONSIGNES GÉNÉRALES

Travail individuel - Toute forme de plagiat sera sanctionnée. L'utilisation intensive de ChatGPT ou autres IA pour générer du code est interdite. Vous pouvez cependant vous en servir pour clarifier des concepts ou obtenir des explications.

Livrables attendus :

1. Code corrigé : **secure.c** (programme sécurisé et fonctionnel)
2. Rapport de sécurité : **RAPPORT.md** (voir template fourni)
3. Makefile fonctionnel

OBJECTIFS PÉDAGOGIQUES

À la fin de ce TP, vous devez être capable de :

- ☒ Identifier les vulnérabilités classiques dans du code C
- ☒ Appliquer les bonnes pratiques de sécurité
- ☒ Remplacer les fonctions dangereuses par leurs équivalents sécurisés
- ☒ Valider et filtrer les entrées utilisateur
- ☒ Gérer correctement la mémoire
- ☒ Utiliser de la cryptographie appropriée

PHASE 1 : ANALYSE

Tâche 1.1 - Lecture et compréhension du code

Lisez attentivement **vulnerable.c** et comprenez son fonctionnement :

- Quelles sont les fonctionnalités ?
- Quels sont les types de données manipulées ?
- Quelles sont les entrées utilisateur ?

Tâche 1.2 - Identification des vulnérabilités

Utilisez la checklist suivante pour identifier **TOUTES** les vulnérabilités :

Checklist de sécurité

A. Buffer Overflows

- ☐ Rechercher tous les usages de **gets()**, **scanf("%s")**, **strcpy()**, **strcat()**, **sprintf()**
- ☐ Vérifier les tailles de buffers dans chaque fonction
- ☐ Identifier les débordements potentiels

B. Validation des entrées

- ☐ Entrées utilisateur validées ?
- ☐ Vérification des indices de tableaux ?
- ☐ Validation des montants, ID, noms de fichiers ?

C. Gestion mémoire

- ☐ Tous les `malloc()` ont un `free()` correspondant ?
- ☐ Fuites mémoire potentielles ?

D. Cryptographie

- ☐ Le hashage est-il sécurisé ?
- ☐ Les mots de passe sont-ils stockés en clair ?
- ☐ Existe-t-il des backdoors/mots de passe codés en dur ?

E. Injection de commandes/Path traversal

- ☐ Utilisation de `system()` ? (La fonction `system()` en C permet d'exécuter une commande shell depuis un programme C.)
- ☐ Ouverture de fichiers avec chemins non validés ?
- ☐ Possibilité d'injection dans les commandes ?

F. Format String

- ☐ Utilisation de `printf(user_input)` ?
- ☐ Format string contrôlé par l'utilisateur ?

G. Logique métier

- ☐ Contrôles d'accès corrects ?
- ☐ Validation des transactions financières ?
- ☐ Protection contre les élévations de privilèges ?

Tâche 1.3 - Documentation des vulnérabilités

Pour chaque vulnérabilité identifiée, notez dans votre rapport :

- Numéro de ligne
- Type de vulnérabilité
- Gravité (Critique / Haute / Moyenne / Faible)
- Description brève

Au moins 25+ vulnérabilités au total



PHASE 2 : CORRECTIONS

Tâche 2.1 - Créer `secure.c`

Copiez `vulnerable.c` vers `secure.c` et commencez les corrections.

```
cp vulnerable.c secure.c
```

Tâche 2.2 - Corrections systématiques

Pour chaque vulnérabilité, appliquez la correction appropriée :

Remplacements de fonctions dangereuses

Fonction dangereuse	Remplacement recommandé
<code>gets(buf)</code>	<code>fgets(buf, size, stdin)</code>
<code>scanf("%s", buf)</code>	<code>scanf("%15s", buf)</code> ou <code>fgets()</code>
<code>strcpy(dst, src)</code>	<code>strncpy(dst, src, size) + null terminator</code>
<code>strcat(dst, src)</code>	<code>strncat(dst, src, size)</code>
<code>sprintf(buf, ...)</code>	<code>snprintf(buf, size, ...)</code>
<code>system(cmd)</code>	Éviter ou valider strictement

Ajout de validations

Créez des fonctions de validation pour :

- Noms d'utilisateur (caractères autorisés, longueur)
- Indices de tableaux ($0 \leq id < \text{max}$)
- Montants (positifs, suffisamment de fonds)
- Noms de fichiers (pas de `..`, `/`, chemins absolus)

Cryptographie

- **À IMPLÉMENTER** : Remplacez `hash_password()` par SHA-256
 - Incluez : `#include <openssl/sha.h>`
 - Utilisez : `SHA256()` de OpenSSL
 - Stockez les hash, pas les mots de passe

Gestion mémoire

- Vérifiez TOUS les retours de `malloc()`
- Ajoutez les `free()` correspondants
- Créez une fonction `cleanup()` complète

Tâche 2.3 - Tests de sécurité

Testez votre code avec des entrées malicieuses :

```
# Tester buffer overflow
echo "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA" | ./secure

# Tester path traversal
echo "../../../etc/passwd" | ./secure

# Tester command injection
echo "ls ; cat /etc/passwd" | ./secure
```

Tâche 2.4 - Vérification mémoire

Utilisez **valgrind** pour détecter les fuites :

```
valgrind --leak-check=full ./secure
```

Objectif : 0 fuite mémoire

PHASE 3 : DOCUMENTATION

Tâche 3.1 - Compléter le rapport

Remplissez le fichier **RAPPORT.md** avec :

Pour **CHAQUE** correction :

1. **Vulnérabilité originale** : Description précise
2. **Attaque possible** : Scénario d'exploitation
3. **Correction implémentée** : Code avant/après
4. **Justification** : Pourquoi la correction est sûre

Tâche 3.2 - Synthèse

Ajoutez une section finale résumant :

- Nombre total de vulnérabilités corrigées
- Difficultés rencontrées
- Améliorations possibles

OUTILS ET RESSOURCES

Compilation

```
# Version vulnérable
gcc -o vulnerable vulnerable.c -w
```

```
# Version sécurisée (avec OpenSSL pour SHA-256)
gcc -o secure secure.c -lcrypto -Wall -Wextra

# Avec protections
gcc -o secure secure.c -lcrypto -fstack-protector-all -D_FORTIFY_SOURCE=2
```

Débogage

```
# Vérification mémoire
valgrind --leak-check=full --show-leak-kinds=all ./secure
```

Documentation recommandée

- [man fgets](#), [man snprintf](#), [man strncpy](#)
- OpenSSL documentation pour SHA-256
- OWASP Top 10



CRITÈRES D'ÉVALUATION DÉTAILLÉS

Analyse

- Identification des buffer overflows
- Identification des problèmes de validation
- Identification des problèmes de crypto
- Identification des injections (command/path)
- Identification des problèmes mémoire

Corrections

- Remplacement de toutes les fonctions dangereuses
- Ajout de validations pertinentes sur les entrées
- Implémentation de SHA-256 correcte
- Gestion mémoire correcte (pas de fuites)
- Code compile sans warnings et fonctionne

Documentation

- Qualité des explications (claires, précises, techniques)
- Exemples d'exploitation fournis
- Justifications des solutions (pourquoi c'est sûr)

BONUS : Implémentation de fonctionnalités de sécurité avancées

- Limitation du nombre de tentatives de connexion
- Salage des mots de passe
- Logs de sécurité
- Protection timing-attack



CONSEILS MÉTHODOLOGIQUES

1. **Commencez par l'analyse** : Ne codez pas avant d'avoir tout identifié
2. **Corrigez par priorité** : Critique > Haute > Moyenne > Faible
3. **Testez régulièrement** : Après chaque correction majeure
4. **Documentez au fur et à mesure** : Ne laissez pas tout pour la fin
5. **Compilez avec -Wall -Wextra** : Le compilateur est votre ami
6. **Utilisez valgrind souvent** : Vérifiez la mémoire régulièrement



LIVRABLES FINAUX

À rendre avant la fin des 4 heures :

```
TP_FINAL/  
├─ vulnerable.c          (fourni, non modifié)  
├─ secure.c             (votre version corrigée)  
├─ RAPPORT.md           (votre documentation)  
├─ Makefile             (pour compiler les deux versions)  
└─ test_inputs.txt      (optionnel: vos tests)
```

BON COURAGE !